Database Systems: Concepts and Design

Lecture 5: Relational Algebra

February 18, 2026

Reading Materials

Database Systems: The Complete Book (2nd edition)

- Chapter 2.1: An Overview of Data Models
- Chapter 2.2: Basics of the Relational Model
- Chapter 2.4: An Algebraic Query Language

Fundamentals of Database Systems (7th edition)

- Chapter 8

Today's Class

1. Relational Algebra: Basic Operators
2. Relational Algebra Pt. II (Derived Operators)
3. Advanced Operators (Outer Joins, Division)
4. Practice: The COMPANY Database

Recall: Data Model

Definition

A notation for describing data or information.

The description generally consists of three parts:

1. **Structure** of the data
2. **Operations** on the data
3. **Constraints** on the data

Adapted from KAIST EE477 (Steven Whang)

1st Part of the Model: Structure of the Data

- Referred to as a "conceptual model" of the data.
- Higher level than "physical data models" or data structures like arrays and lists.

sid	name	gpa
001	Bob	3.2
002	Joe	2.8
003	Mary	3.8
004	Alice	3.5

Example: A relation consists of a schema, attributes, and tuples.

The Relational Model: Schema

Relational Schema

Students(sid: string, name: string, gpa: float)

- **Relation name:** Students
- **Attributes:** sid, name, gpa
- **Domains:** String, float, int, etc. (the types of the attributes)

The Relational Model: Data

Student

sid	name	gpa
001	Bob	3.2
002	Joe	2.8
003	Mary	3.8
004	Alice	3.5

An **attribute** (or column) is a typed data entry present in each tuple.

The Relational Model: Data

Student

sid	name	gpa
001	Bob	3.2
002	Joe	2.8
003	Mary	3.8
004	Alice	3.5

A **tuple** or row (or record) is a single entry in the table having the attributes specified by the schema.

The Relational Model: Data

Student

sid	name	gpa
001	Bob	3.2
002	Joe	2.8
003	Mary	3.8
004	Alice	3.5

- A **relational instance** is a set of tuples all conforming to the same schema.
- *Recall:* In practice, DBMSs relax the set requirement and use **multisets** (duplicates allowed).

Relational Schema vs. Relational Instance

- A **relational schema** describes the data that is contained in a relational instance.
- Let $R(f_1 : Dom_1, \ldots, f_m : Dom_m)$ be a relational schema.
- Then, an **instance** of R is a subset of:

$$Dom_1 \times Dom_2 \times \cdots \times Dom_m$$

- In this way, a relational schema R is a total function from attribute names to types.

A Relational Database

- A **relational database schema** is a set of relational schemata, one for each relation.
- A **relational database instance** is a set of relational instances, one for each relation.

Two conventions:

1. We call relational database instances simply **databases**.
2. We assume all instances are **valid**, i.e., they satisfy the domain constraints.

Example

Relational DB Schema:

- Students(sid: string, name: string, gpa: float)
- Courses(cid: string, cname: string, credits: int)
- Enrolled(sid: string, cid: string, grade: string)

Relation Instances:

Students

sid	name	gpa
101	Bob	3.2
123	Mary	3.8

Courses

cid	cname	credits
564	564-2	4
308	417	2

Enrolled

sid	cid	grade
123	564	A

Note: Schemas impose effective domain/type constraints (e.g., GPA can't be "Apple").

2nd Part of the Model: Operations

Usually a limited set of operations that can be performed:

- **Queries:** Operations that retrieve information.
- **Modifications:** Operations that change the database.

Strength, not a weakness

- Programmers can describe operations at a very high level.
- The DBMS implements them efficiently.
- *Not easy to do when coding in C/C++ directly.*

The Relational Model: Operations

"Find names of all students with GPA > 3.5*"*

```
SELECT S.name
FROM Students S
WHERE S.gpa > 3.5;
```

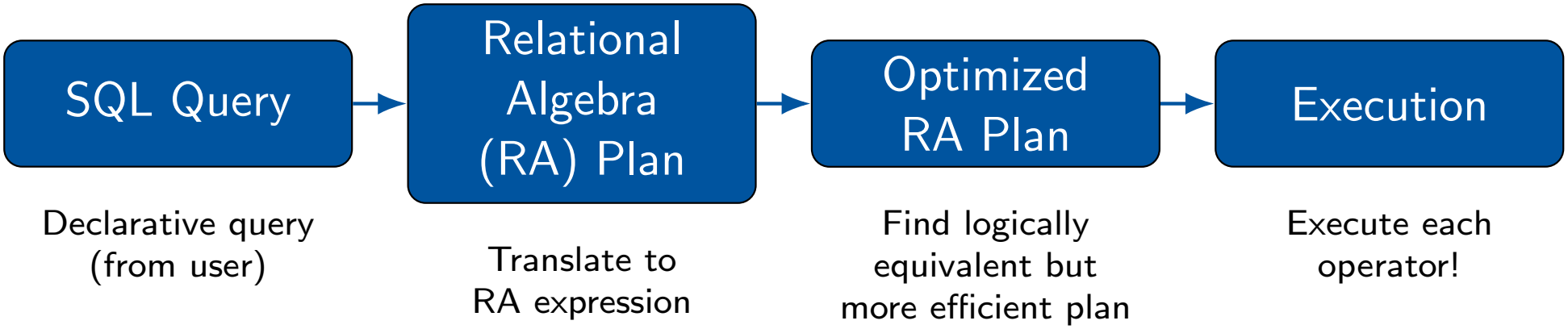
We specify **how** or **where** to get the data – just **what** we want. **Querying is declarative.**

To make this happen, we need to translate the declarative query into a series of operators... **Relational Algebra**.

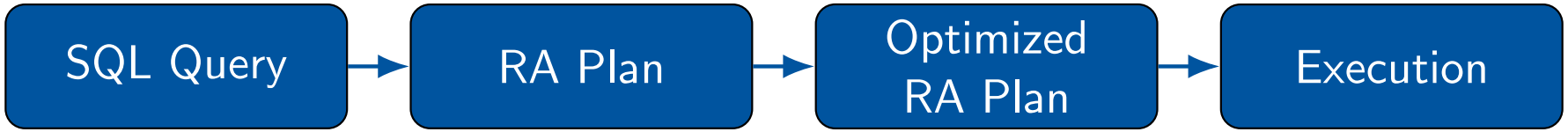
1. Relational Algebra

The Big Picture: RDBMS Architecture

How does a SQL engine work?



The Big Picture: RDBMS Architecture



Relational Algebra allows us to translate declarative (SQL) queries into precise and **optimizable** expressions!

Relational Algebra (RA)

Five basic operators:

1. Selection: σ
2. Projection: π (or Π)
3. Cartesian Product: $\times$
4. Union: $\cup$
5. Difference: $-$

We'll look at these first!

Derived or auxiliary operators:

- Intersection, complement
- Joins (natural, equi-join, theta join, semi-join)
- Renaming: ρ
- Grouping: γ
- **Outer Joins, Division**

Note: RA Operates on Sets!

- RDBMSs use **multisets** (bags), however in relational algebra formalism we will consider **sets**!
- Also: we will consider the **named perspective**, where every attribute must have a unique name.
  $\rightarrow$ Attribute order does not matter.

Let's go to the basic RA operators...

1. Selection (σ)

- Returns all tuples which satisfy a condition.
- **Notation:** $\sigma_c(R)$
- **Examples:**
  - $\sigma_{\text{Salary}>40000}(\text{Employee})$
  - $\sigma_{\text{name}=\text{"Smith"}}(\text{Employee})$
- The condition c can be $=, <, \leq, >, \geq, \neq$.

Students(sid, sname, gpa)

SQL:

```
SELECT *
FROM Students
WHERE gpa > 3.5;
```

RA: $\sigma_{\text{gpa}>3.5}(\text{Students})$

Selection Example

Employee Table

SSN	Name	Salary
123	John	200000
542	Smith	600000
435	Fred	500000

Query: $\sigma_{\text{Salary}>400000}(\text{Employee})$

SSN	Name	Salary
542	Smith	600000
435	Fred	500000

2. Projection (π)

- Eliminates columns, then **removes duplicates**.
- **Notation:** $\pi_{A_1,\ldots,A_n}(R)$
- **Example:** Project SSN and Name:
- $\pi_{\text{SSN, Name}}(\text{Employee})$
- Output schema: Answer(SSN, Name)

Students(sid, sname, gpa)

SQL:
```
SELECT DISTINCT
sname, gpa FROM
Students;
```

RA: $\pi_{\text{sname, gpa}}(\text{Students})$

Projection Example

Employee Table

SSN	Name	Salary
123	John	200000
542	John	600000
435	John	200000

Query: $\pi_{\text{Name, Salary}}(\text{Employee})$

Name	Salary
John	200000
John	600000

(Note: Duplicate (John, 200000) removed)

RA Operators are Compositional

Students(sid, sname, gpa)

Query:

```
SELECT DISTINCT sname, gpa
FROM Students
WHERE gpa > 3.5;
```

How do we represent this in RA?

$$\pi_{\text{sname, gpa}}(\sigma_{\text{gpa}>3.5}(\text{Students}))$$

Is this equivalent?

$$\sigma_{\text{gpa}>3.5}(\pi_{\text{sname, gpa}}(\text{Students}))$$

RA Operators are Compositional

Students(sid, sname, gpa)

Query:

```
SELECT DISTINCT sname, gpa
FROM Students
WHERE gpa > 3.5;
```

How do we represent this in RA?

$$\pi_{\text{sname, gpa}}(\sigma_{\text{gpa}>3.5}(\text{Students}))$$

Is this equivalent?

$$\sigma_{\text{gpa}>3.5}(\pi_{\text{sname, gpa}}(\text{Students}))$$

Think about it: The second one might fail if 'gpa' is not in the projection list!

3. Cross-Product ($\times$)

- Each tuple in R1 paired with each tuple in R2.
- **Notation:** $R1 \times R2$
- **Example:** Employee $\times$ Dependents
- Rare in practice on its own; mainly used to express joins.

SQL: `SELECT * FROM Students, People;`
RA: Students $\times$ People

Cross-Product Example

People

ssn	pname	address
123	John	216 Rosse
542	Bob	217 Rosse

Students

sid	sname	gpa
001	John	3.4
002	Bob	1.3

$\times$

Students $\times$ People

ssn	pname	addr	sid	sname	gpa
123	John	...	001	John	3.4
542	Bob	...	001	John	3.4
123	John	...	002	Bob	1.3
542	Bob	...	002	Bob	1.3

Renaming (ρ)

- Changes the schema, not the instance.
- A 'special' operator - neither basic nor derived.
- **Notation:** $\rho_{B_1,\ldots,B_n}(R)$
- Note: this is shorthand for $\rho_{A_1 \to B_1,\ldots}(R)$.

SQL:

```
SELECT sid AS studId, sname AS name, gpa AS gradePtAvg
FROM Students;
```

RA: $\rho_{\text{studId, name, gradePtAvg}}(\text{Students})$

Renaming Example

Students

sid	sname	gpa
001	John	3.4
002	Bob	1.3

$$\rho_{\text{studId, name, gradePtAvg}}(\text{Students})$$

Result

studId	name	gradePtAvg
001	John	3.4
002	Bob	1.3

Natural Join ($\bowtie$)

- $R_1 \bowtie R_2$: Joins R_1 and R_2 on equality of **all shared attributes**.
- If R_1 has attributes A, R_2 has B, and shared $C = A \cap B$.
- **Definition:**

$$R_1 \bowtie R_2 = \pi_{A \cup B}(\sigma_{R_1.C = R_2.C}(R_1 \times R_2))$$

SQL:

```
SELECT DISTINCT...
FROM Students S, People P
WHERE S.name = P.name;
```

RA: Students $\bowtie$ People

Natural Join Example

Students S (shared: name)

sid	**name**	gpa
001	John	3.4
002	Bob	1.3

People P (shared: name)

ssn	**name**	addr
123	John	216
542	Bob	217

Result: $S \bowtie P$

sid	**name**	gpa	ssn	addr
001	John	3.4	123	216
002	Bob	1.3	542	217

Matches on 'name'.

Example: Converting SFW Query → RA

Schema: Students(sid, name, gpa), People(ssn, name, address)

SQL Query

```
SELECT DISTINCT gpa, address
FROM Students S, People P
WHERE gpa > 3.5 AND S.name = P.name;
```

How do we represent this in RA?

$$\pi_{\text{gpa, address}}(\sigma_{\text{gpa}>3.5 \wedge S.\text{name}=P.\text{name}}(\text{Students} \times \text{People}))$$

In Class Exercise

- Given schemas $R(A, B, C, D)$, $S(A, C, E)$, what is the schema of $R \bowtie S$?
- Given $R(A, B)$, $S(A, B)$, what is $R \bowtie S$?
- Given $R(A, B, C)$, $S(D, E)$, what is $R \bowtie S$?

2. Relational Algebra Pt. II

Relational Algebra (RA) Review

Five basic operators:

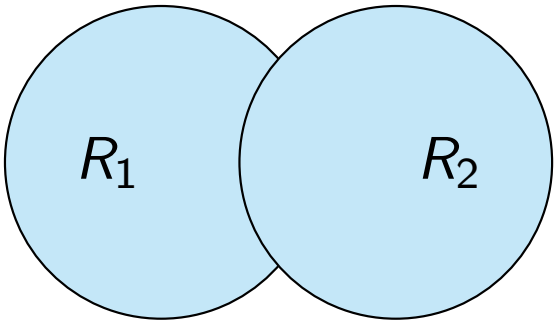
1. Selection: σ
2. Projection: π
3. Cartesian Product: $\times$
4. Union: $\cup$
5. Difference: $-$

Derived operators we will cover:

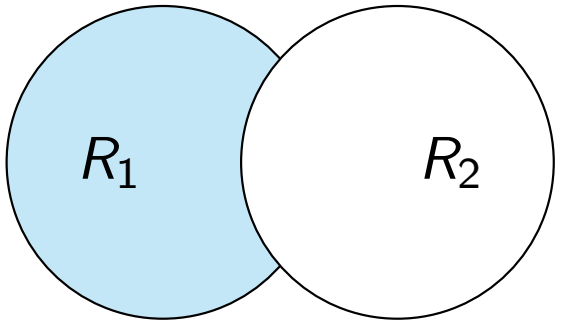
- Intersection: $\cap$
- Joins: $\bowtie_\theta$, $\ltimes$, etc.
- Grouping: γ
- **Outer Joins, Division**

Union ($\cup$) and Difference ($-$)

- $R_1 \cup R_2$: All tuples in R1 or R2.
  - Example: ActiveEmployees $\cup$ RetiredEmployees
- $R_1 - R_2$: Tuples in R1 but NOT in R2.
  - Example: AllEmployees $-$ RetiredEmployees



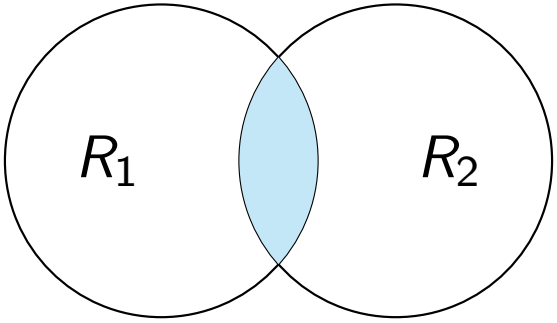
Union



Difference ($R_1 - R_2$)

Intersection ($\cap$)

- Derived operator: $R_1 \cap R_2 = R_1 - (R_1 - R_2)$
- Also expressed as a join!
- Example: UnionizedEmployees $\cap$ RetiredEmployees



In Class Exercise

Suppose relations R and S have n and m tuples, respectively. What are the **minimum** / **maximum** number of tuples of the following expressions?

1. $R \cup S$
2. $R \bowtie S$ (Natural Join)
3. $\pi_L(R) - S$ (for some attribute list L)

Theta Join ($\bowtie_\theta$)

- A join that involves a predicate θ.
- $R_1 \bowtie_\theta R_2 = \sigma_\theta(R_1 \times R_2)$
- θ can be any condition.

SQL: `SELECT * FROM Students, People`
`WHERE` θ`;`

Note that natural join is a theta join + a projection (to remove duplicate columns).

Theta Join Example

R

A	B
1	2
7	2
8	4

S

B	C
2	5
4	7
4	9

$R \bowtie_{A<C} S$

A	R.B	S.B	C
1	2	2	5
1	2	4	7
1	2	4	9
7	2	4	9
8	4	4	9

Theta Join Example 2

Predicate: $A < C$ AND $R.B \neq S.B$

A	R.B	S.B	C
1	2	2	5
1	2	4	7
1	2	4	9
7	2	4	9
8	4	4	9

(Rows in red filtered out by $R.B \neq S.B$)

Equi-Join ($\bowtie_{A=B}$)

- A theta join where θ is an equality.
- $R_1 \bowtie_{A=B} R_2 = \sigma_{A=B}(R_1 \times R_2)$
- **Example:** Employee.SSN = Dependents.SSN
- *Most common join in practice!*

RA: $S \bowtie_{\text{sname}=\text{pname}} P$

Semijoin ($\ltimes$)

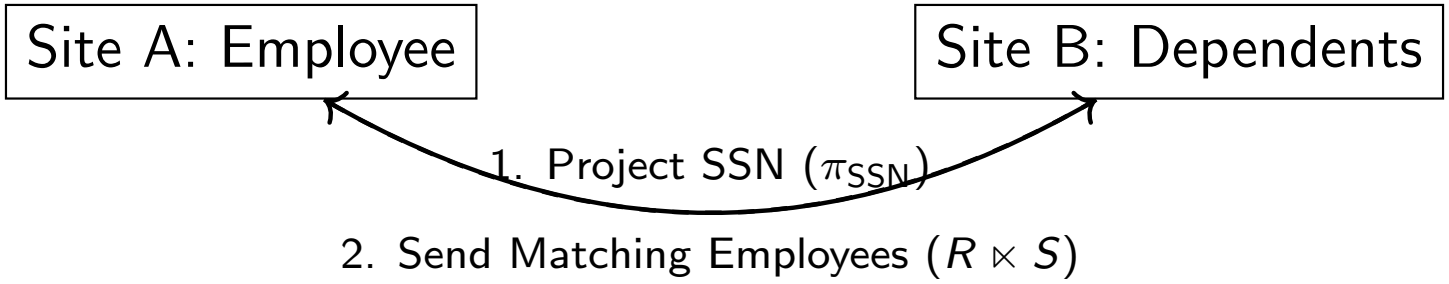
- $R \ltimes S = \pi_{\text{Attributes}(R)}(R \bowtie S)$
- "Find tuples in R that match a tuple in S".
- **Example:** Students $\ltimes$ People
- **SQL:**

```
SELECT DISTINCT sid, sname, gpa
FROM Students, People
WHERE sname = pname;
```

Semijoins in Distributed Databases

Semijoins are often used to compute natural joins in distributed databases.

Goal: Send less data to reduce network bandwidth!



Outer Joins

- **Left Outer Join ($R \bowtie\!\!\!\mid S$):** Keeps all tuples in R. If no match in S, pads with NULL.
- **Right Outer Join ($R \mid\!\!\bowtie S$):** Keeps all tuples in S.
- **Full Outer Join ($R \mid\!\!\bowtie\!\!\mid S$):** Keeps all tuples from both.

Student (S)

Sid	Name
1	Bob
2	Joe

Enrolled (E)

Sid	Cid
1	CS101
3	CS102

Result: S Left Outer Join E

Sid	Name	Cid
1	Bob	CS101
2	Joe	NULL

Joe has no courses, but is kept.

The Division Operator ($\div$)

- Also known as the "For All" query.
- **Intuition:** $R(A, B) \div S(B)$ returns values of A from R that are associated with **every** value in S.
- **Use Case:** "Find students who have taken **all** Biology courses."

Taken(Student, Course)

S	C
Joe	Bio1
Joe	Bio2
Bob	Bio1

$\div$

Required(Course)

C
Bio1
Bio2

$=$

Result

Student
Joe

Bob is missing Bio2, so he is not in the result.

Grouping (γ)

- The grouping operator γ consists of:
  - **Grouping attributes:** attributes to group by.
  - **Aggregation attributes:** applied operations (SUM, AVG, MIN, MAX, COUNT).

R

A	B	C
1	1	1
1	2	3
5	2	3

$$\gamma_{A,MIN(B)\rightarrow minB,AVG(C)\rightarrow avgC}(R)$$

Result

A	minB	avgC
1	1	2
5	2	3

Aside: Relational Calculus

- We have studied **Relational Algebra**, which is **procedural**.
  - We specify the steps: Join A and B, then Filter by X.
- **Relational Calculus** is a **declarative** alternative.
  - Based on First-Order Logic (Predicate Logic).
  - We describe "what" we want, not "how" to compute it.
  - **Tuple Relational Calculus (TRC):** Variables range over tuples.
  - **Domain Relational Calculus (DRC):** Variables range over attribute domains.
- **Why does it matter?**
  - SQL is largely based on Relational Calculus (declarative).
  - The DBMS translates SQL (Calculus) $\rightarrow$ RA (Algebra) for execution!

Combining Operations to Form Queries

- RA expressions can be arbitrarily complicated.
- Multiple RA expressions may be equivalent.

Example 1:

$$\pi_{A,D}(\sigma_{A<10}(T \times (R \bowtie S)))$$

Example 2:

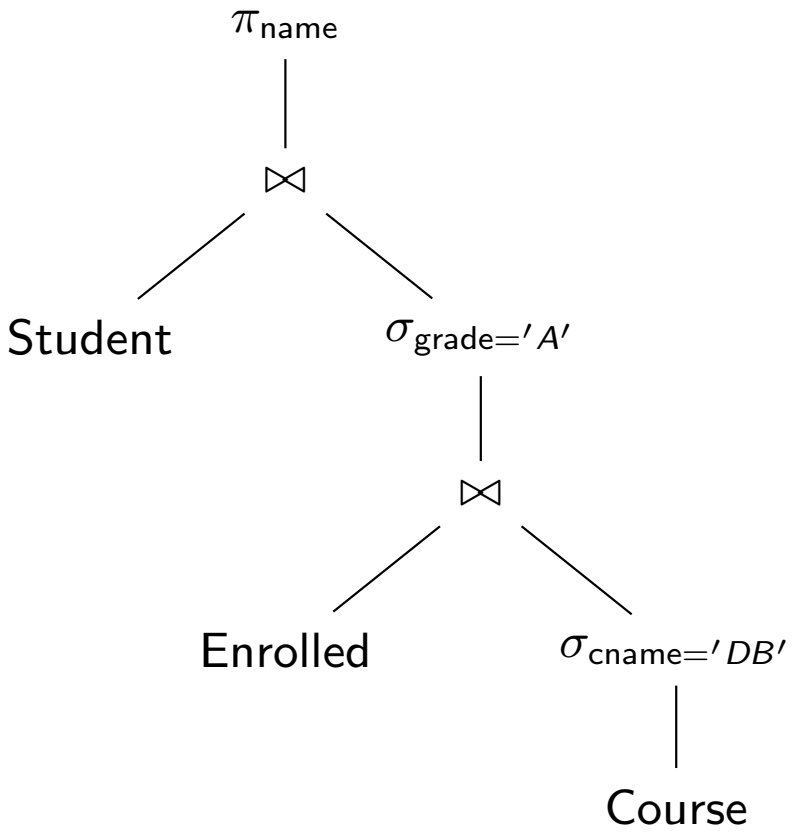
$$\pi_{A,D}(T \times \pi_{A,C}(\sigma_{A<10}(R) \bowtie S))$$

Which version is more efficient? *Logical optimization: Find equivalent RA expressions that are more efficient.*

Complex Query Walkthrough

Query: "Find names of students who got an 'A' in 'Database Systems'."

Schema:
Student(id, name)
Enrolled(sid, cid, grade)
Course(cid, cname)



Step-by-Step:

1. Filter Course for 'DB'.
2. Join with Enrolled (Natural Join on cid).
3. Filter result for grade 'A'.
4. Join with Student (Natural Join on sid).
5. Project only the name.

Part 5: Practice with Standard Schema (COMPANY Database)

Reference Schema: COMPANY Database

(From Elmasri & Navathe, Chapter 8)

- **EMPLOYEE**(Ssn, Fname, Lname, Salary, Super_ssn, Dno)
- **DEPARTMENT**(Dnumber, Dname, Mgr_ssn)
- **PROJECT**(Pnumber, Pname, Plocation, Dnum)
- **WORKS_ON**(Essn, Pno, Hours)
- **DEPENDENT**(Essn, Dep_name, Relationship)

Questions

Try to solve the following queries on paper before advancing to the next slide to see the solution!

Practice 1: Basic Join

Query: "Retrieve the name and address of all employees who work for the 'Research' department."

Practice 1: Basic Join

Query: "Retrieve the name and address of all employees who work for the 'Research' department."

Strategy:

1. Select 'Research' from DEPARTMENT.
2. Join with EMPLOYEE (on Dnumber = Dno).
3. Project First Name, Last Name, Address.

Solution:

$$\pi_{\text{Fname, Lname, Address}}(\sigma_{\text{Dname}='\textit{Research}'}(\text{DEPARTMENT} \bowtie_{\text{Dnumber}=\text{Dno}} \text{EMPLOYEE}))$$

Practice 2: Self-Join (Important!)

Query: "For every employee, find their name and their supervisor's name."

Hint: You need to join EMPLOYEE with itself! Use **Renaming** *(ρ).*

Practice 2: Self-Join (Important!)

Query: "For every employee, find their name and their supervisor's name."

Hint: You need to join EMPLOYEE with itself! Use **Renaming** *(ρ).*

Strategy:

1. Rename EMPLOYEE to E (Subordinates).
2. Rename EMPLOYEE to S (Supervisors).
3. Join E and S where $E.\text{Super_ssn} = S.\text{Ssn}$.

Solution:

$$\pi_{E.\text{Lname},S.\text{Lname}}(\rho_E(\text{EMPLOYEE}) \bowtie_{E.\text{Super_ssn}=S.\text{Ssn}} \rho_S(\text{EMPLOYEE}))$$

Practice 3: Set Operations

Query: "Find the Ssn of employees who have no dependents."

Practice 3: Set Operations

Query: "Find the Ssn of employees who have no dependents."

Strategy:

- Get all employee Ssns.
- Get Ssns from Dependent table (employees who HAVE dependents).
- Subtract the second set from the first.

Solution:

$$\pi_{\text{Ssn}}(\text{EMPLOYEE}) - \pi_{\text{Essn}}(\text{DEPENDENT})$$

Alternatively, you could use a Left Outer Join and filter for NULLs!

Practice 4: Aggregation

Query: "For each department, retrieve the department number and the number of employees in that department."

Practice 4: Aggregation

Query: "For each department, retrieve the department number and the number of employees in that department."

Strategy:

1. Group by Department Number (*Dno*).
2. Count the number of Ssns in each group.

Solution:

$$_{\text{Dno}}\mathfrak{I}_{\text{COUNT(Ssn)}}(\text{EMPLOYEE})$$